

G-2 Live Boot QA

QA Evidence — Live helixcode boot + end-to-end user flow

Run-id: g2-live-boot-20260613 **Date:** 2026-06-13 **Feature:** helixcode self-boot infra (HEAD 8f901bc2) + end-to-end REST API against real Postgres/Redis
Operator request: “boot the system - run helixcode binary - continue” **Anti-bluff basis:** §11.4.5 / §11.4.13 (sink-side) / §11.4.69 (boot_service) / §11.4.83 / §11.4.108 (user-visible). All HTTP codes + Postgres rows are captured from a real run this session — no metadata-only / simulated results.

1. Pre-flight (§11.4.82(C))

- Binary: bin/helixcode (46 MB) present.
- Host memory: 44% free / 56% used (under §12.6 60% ceiling).
- podman-machine-default: running (4 GiB, 150 GiB disk).
- Ports 55432 / 56379: free.
- HELIX_AUTOBOOT_INFRA: unset (defaults on).

2. Self-boot (./bin/helixcode, PID 67282) — captured log

```
2026/06/13 12:41:30 ✅ Infra auto-boot: podman booted
postgres:55432 redis:56379
2026/06/13 12:41:30 ✅ Database connection established
successfully
2026/06/13 12:41:30 ✅ Database schema already exists
2026/06/13 12:41:30 🚀 Starting HelixCode server on 0.0.0.0:8080
```

Real podman containers (both **healthy**):

helixcode-autoboot-postgres	postgres:15	Up (healthy)
0.0.0.0:55432->5432/tcp		
helixcode-autoboot-redis	redis:7-alpine	Up (healthy)
0.0.0.0:56379->6379/tcp		

Memory after boot: 45% free / 55% used (under ceiling).

3. Health + auth gating (captured)

Request	Result
GET /health	200 {"status":"healthy","version":"1.0.0"}
GET /api/v1/health	200 healthy
GET /api/v1/tasks (no auth)	401 (auth middleware live)
GET /api/v1/llm/models	200 count=7, verifier-sourced models (CONST-036)
GET /api/v1/llm/providers	200 7 providers

4. End-to-end user flow — API + direct Postgres (sink-side §11.4.13)

Step	API result	Real Postgres (helixcode_prod, direct psql)
POST /api/v1/auth/register	201 user b1148d17-...	users: demo_1781345386 \ ... @example.com \ active=t
POST /api/v1/auth/login	200 JWT (283 chars)	user_sessions count = 2
POST /api/v1/tasks (authed)	201 task fc244a37-...	distributed_tasks: fc244a37 \ pending \ 10
GET /api/v1/tasks/:id	200 same task	—
POST /api/v1/tasks/:id/start	200 “started”	status pending → running
POST /api/v1/tasks/:id/complete (body)	200 “completed”	status running → completed

Conclusion: the booted system persists real users/sessions/tasks to real Postgres and runs the full task lifecycle — confirmed by direct DB queries, not just API responses. The DB-backed `distributed_tasks` table (not an in-memory map) holds the task; state transitions are durable.

5. Honest observations (§11.4.6 — not bluffed)

- `llm_models` / `llm_providers` DB tables are **empty (0 rows)** while `/api/v1/llm/models` serves 7 — the catalog is served from the **LLMsVerifier cache**

(**CONST-036 single-source-of-truth**), not the DB. Consistent with CONST-036, not a defect.

- Real LLM generation (BLUFF-001 path) **NOT** exercised — Ollama not running; starting a ~2–3 GB model at 55% memory used would risk the §12.6 60% ceiling. Operator chose “hold” rather than start it.

6. API-surface map (subagent, read-only)

26 production GET routes probed against the live server: - **23 live + working** — 9 public 200 (/health, /api/v1/health, /server/info, /metrics, /llm/{providers,models,providers/:id}, /memory/{systems,stats}) + 14 auth-gated (401 unauthenticated → 200/correct-404 with Bearer token: /users/me, /tasks*, /projects*, /sessions*, /workers*, /system/{stats,status}). - **3 feature-disabled-by-config** — /qa/sessions, /qa/session/:id/status, /screenshot/engines return 503 (intentional graceful degradation, guarded by `qaEngine == nil || !Enabled()` in `qa_handlers.go:18–19`; QA engine not booted in this deployment — not a defect). - **0 real 5xx defects**. Server version 1.0.0, database.connected:true, uptime ~33m. Doc note: current-user route is /api/v1/users/me (not /api/v1/auth/me).

7. Security probe (subagent, non-destructive)

All controls held — **0 security findings**: - Unauthenticated access to /tasks, /projects, POST /tasks, /users/me → all 401 (Authorization header required). - JWT tampering: flipped-signature → 401 signature is invalid; invalid.token.here → 401; **alg=none forgery (3 variants) → 401 unexpected_signing_method** (canonical alg-confusion defense). - SQL-injection-style login (admin' OR '1'='1, ' OR 1=1—, SQLi in password) → clean 401 invalid credentials, **no bypass, no 500** (parameterized/ORM query path). - Wrong password for valid user → 401.

8. §11.4.79 CodeGraph index — config-confirmed, runtime probe env-blocked (honest)

- **Scope inclusion CONFIRMED at config level**: .codegraph/config.json includes `**/*.go` with no `submodules/**` exclusion → `submodules/dag_orchestrator` + `submodules/pipeline_runtime` are in index scope per §11.4.79.
- **Runtime symbol-resolution probe NOT run (env-blocked, not faked)**: lumen’s embedding backend is Ollama on :11434, which is **uninstalled** (dangling /usr/local/bin/ollama symlink, `ollama.app` + `~/.ollama` removed; LM Studio also removed). No embedding backend exists, so `semantic_search` returns “all embedding servers are unhealthy”. The subagent correctly did NOT fabricate a probe result (§11.4.6/§11.4.123). Re-validate after an operator reinstalls an embedding backend + pulls `ordis/jina-embeddings-v2-base-code`.
- **Resource note (operator)**: a stuck `codegraph sync` (PID 36072) has held `.codegraph/codegraph.lock` ~47 min with a 251 MB growing WAL (index

DB 4.4 GB at repo root) — likely hung because the embedding backend is down; worth killing to free resources + the lock.

Verdict

The booted helixcode system is **functional, persistent, and secure** for the end user: full auth + task lifecycle against real Postgres (sink-side verified), 23/26 API routes live with 0 defects, all auth/security controls held. Only the LLM-generation path (Ollama) and the CodeGraph runtime probe are environmentally unavailable (Ollama uninstalled) — both honestly reported, neither faked.